SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1-EXPERIMENTS

Course Code:	**CSA1209 & CSA1215**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO3**	**Assessment:**	**ASSESSMENT TOOL 1-EXPERIMENTS**
Weightage:	**40%**	**Total Marks:**	**25**
CO3 Statement:	*Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)*		
Student Name:		**Reg.No.:**	
Department:		**Date:**	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I V. Reddy Sreya *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts(20%)	**Experimental Design(20%)**	**Use of Tools(25%)**	**Analysis of Results(20%)**	**Documentation(15 %)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Register Transfer and ALU Operations

Goal: Demonstrate how data moves between registers and how the ALU performs arithmetic and logic operations.

Example registers:

- R1 = 10
- R2 = 5
- R3 = 0
- R4 = 0

Example sequence:

R3 ← R1 + R2
R4 ← R1 - R2
R3 ← R3 AND R2
R4 ← R3 OR R1

The **ALU** performs:

Operation	Input	Output
ADD	R1 + R2	15
SUB	R1 - R2	5
AND	R3 AND R2	Depends on binary values
OR	R3 OR R1	Depends on binary values

Simulation should show:

Initial:
R1 = 10
R2 = 5
R3 = 0
R4 = 0

Step 1: R3 ← R1 + R2
R3 = 15

Step 2: R4 ← R1 - R2
R4 = 5

Step 3: R3 ← R3 AND R2

R3 = 5

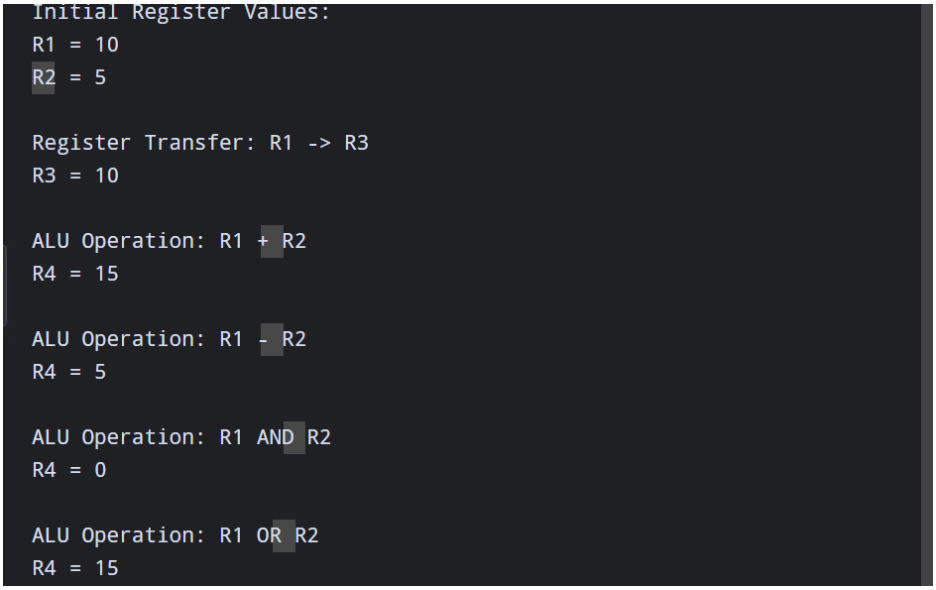
Step 4: R4 ← R3 OR R1
R4 = 15

Code:

```
#include <iostream>
using namespace std;

int main() {
    int R1 = 10;
    int R2 = 5;
    int R3 = 0;
    int R4 = 0;

    cout << "Initial Register Values:\n";
    cout << "R1 = " << R1 << endl;
    cout << "R2 = " << R2 << endl;

    // Register Transfer
    R3 = R1;
    cout << "\nRegister Transfer: R1 -> R3" << endl;
    cout << "R3 = " << R3 << endl;

    // ALU Addition
    R4 = R1 + R2;
    cout << "\nALU Operation: R1 + R2" << endl;
    cout << "R4 = " << R4 << endl;

    // ALU Subtraction
    R4 = R1 - R2;
    cout << "\nALU Operation: R1 - R2" << endl;
    cout << "R4 = " << R4 << endl;

    // AND
    R4 = R1 & R2;
    cout << "\nALU Operation: R1 AND R2" << endl;
    cout << "R4 = " << R4 << endl;

    // OR
    R4 = R1 | R2;
    cout << "\nALU Operation: R1 OR R2" << endl;
    cout << "R4 = " << R4 << endl;
```

```
    return 0;
}
```

Output:



Main concept: Registers store data, the bus transfers data, and the ALU processes the data**.**

2. Single-Bus vs Multi-Bus Organization

Here you model how a CPU transfers data between:

Registers ↔ Bus ↔ ALU ↔ Memory
↕
I/O

Single-bus organization

Only one major data path is available.

For example:

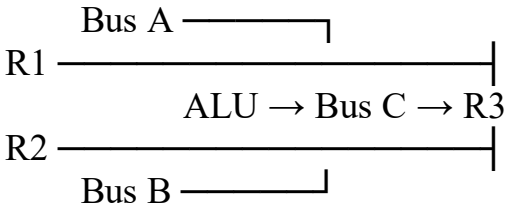
R1 → Bus → ALU
R2 → Bus → ALU
ALU → Bus → R3

Because only one transfer can use the bus at a time, more clock cycles may be required.

Multi-bus organization

Multiple buses allow simultaneous transfers.

For example:



Now two operands can reach the ALU simultaneously.

Comparison

Feature	Single Bus	Multi Bus
Number of buses	1	2 or more
Hardware complexity	Low	Higher
Data transfer	Mostly sequential	More parallel
Clock cycles	More	Fewer
Bottleneck	Higher	Lower
Performance	Lower	Higher

Code:

```
#include <iostream>

using namespace std;


int main() {

    int R1 = 10;

    int R2 = 20;

    int R3 = 0;
```

```
cout << "===== SINGLE-BUS ORGANIZATION =====\n";

int singleBusCycles = 0;

// R1 -> Bus -> R3
cout << "R1 -> Bus -> R3" << endl;
R3 = R1;
singleBusCycles++;

// R2 -> Bus -> R3
cout << "R2 -> Bus -> R3" << endl;
R3 = R2;
singleBusCycles++;

// R1 + R2 -> R3
cout << "R1 + R2 -> R3" << endl;
R3 = R1 + R2;
singleBusCycles += 2;

cout << "Final R3 = " << R3 << endl;
cout << "Single-Bus Clock Cycles = "
     << singleBusCycles << endl;
```

```
cout << "\n===== MULTI-BUS ORGANIZATION =====\n";

R3 = 0;
int multiBusCycles = 0;

// Two registers can be supplied simultaneously
cout << "R1 and R2 transferred simultaneously to ALU" << endl;

R3 = R1 + R2;
multiBusCycles++;

cout << "ALU Result -> R3" << endl;
cout << "Final R3 = " << R3 << endl;

multiBusCycles++;

cout << "Multi-Bus Clock Cycles = "
     << multiBusCycles << endl;


cout << "\n===== COMPARISON =====\n";
```

```
    cout << "Single Bus Cycles = "
         << singleBusCycles << endl;

    cout << "Multi Bus Cycles = "
         << multiBusCycles << endl;

    cout << "Multi-bus organization reduces "
         << "data transfer bottlenecks." << endl;

    return 0;
}
```

Output:

```
===== SINGLE-BUS ORGANIZATION =====
R1 -> Bus -> R3
R2 -> Bus -> R3
R1 + R2 -> R3
Final R3 = 30
Single-Bus Clock Cycles = 4

===== MULTI-BUS ORGANIZATION =====
R1 and R2 transferred simultaneously to ALU
ALU Result -> R3
Final R3 = 30
Multi-Bus Clock Cycles = 2

===== COMPARISON =====
Single Bus Cycles = 4
Multi Bus Cycles = 2
Multi-bus organization reduces data transfer bottlenecks.
```

For your simulation, measure:

Number of clock cycles
Total execution time
Number of transfers
Data-transfer efficiency

3. Five-Stage Instruction Pipeline

Simulate these five stages:

IF → ID → EX → MEM → WB

where:

- **IF** = Instruction Fetch
- **ID** = Instruction Decode
- **EX** = Execute
- **MEM** = Memory Access
- **WB** = Write Back

Suppose we have:

I1: ADD R1,R2,R3
I2: SUB R4,R5,R6
I3: AND R7,R8,R9
I4: OR R10,R11,R12

Pipeline execution can be represented as:

Cycle	**I1**	**I2**	**I3**	**I4**
1	IF			
2	ID	IF		
3	EX	ID	IF	
4	MEM	EX	ID	IF
5	WB	MEM	EX	ID
6		WB	MEM	EX
7			WB	MEM
8				WB

The important observation is that multiple instructions are being processed simultaneously, but each is in a different stage.

Performance

Compare:

Non-pipelined:
I1 → I2 → I3 → I4

Pipelined:
I1
  I2
    I3
      I4

Calculate:

Speedup

Speedup = Non-pipelined execution time

Pipelined execution time

Throughput

Throughput = Number of completed instructions

Total execution time

Code:

```
#include <iostream>
using namespace std;

int main() {

    int instructions = 6;
    int stages = 5;

    // Non-pipelined execution
    int nonPipelinedCycles = instructions * stages;

    // Pipelined execution
    int pipelinedCycles = stages + instructions - 1;
```

```
    double speedup =
        (double)nonPipelinedCycles / pipelinedCycles;

    double throughput =
        (double)instructions / pipelinedCycles;

    cout << "===== 5-STAGE PIPELINE =====\n\n";

    cout << "Instructions = " << instructions << endl;

    cout << "\nPipeline Stages:\n";
    cout << "1. IF  - Instruction Fetch\n";
    cout << "2. ID  - Instruction Decode\n";
    cout << "3. EX  - Execute\n";
    cout << "4. MEM - Memory Access\n";
    cout << "5. WB  - Write Back\n";

    cout << "\nNon-Pipelined Cycles = "
         << nonPipelinedCycles << endl;

    cout << "Pipelined Cycles = "
         << pipelinedCycles << endl;

    cout << "Speedup = "
         << speedup << endl;

    cout << "Throughput = "
         << throughput
         << " instructions/cycle" << endl;

    cout << "\nPipeline improves instruction throughput."
         << endl;

    return 0;
}
```

Output:

```
===== 5-STAGE PIPELINE =====

Instructions = 6

Pipeline Stages:
1. IF  - Instruction Fetch
2. ID  - Instruction Decode
3. EX  - Execute
4. MEM - Memory Access
5. WB  - Write Back

Non-Pipelined Cycles = 30
Pipelined Cycles = 10
Speedup = 3
Throughput = 0.6 instructions/cycle

Pipeline improves instruction throughput.
```

4. Pipeline Hazards

This topic extends Topic 3 by introducing problems that occur when instructions overlap.

There are three major hazards.

A. Data Hazard

Example:

I1: R1 ← R2 + R3
I2: R4 ← R1 + R5

I2 needs R1, but I1 has not finished writing it yet.

Possible solutions:

Stalling

I1: IF ID EX MEM WB
I2: IF ID ST ST EX MEM WB

Data forwarding

Instead of waiting for the value to be written back to the register, forward the result directly from one pipeline stage to another.

B. Control Hazard

Occurs with branch instructions.

Example:

I1: ADD R1,R2,R3
I2: BEQ R1,R0,LABEL
I3: SUB R4,R5,R6

The processor doesn't immediately know whether I3 should execute.

A solution is branch prediction**.**

Branch prediction:
Taken / Not Taken

The processor predicts the branch and continues fetching instructions.

C. Structural Hazard

Occurs when two instructions require the same hardware resource at the same time.

Example:

Instruction 1 → Memory
Instruction 2 → Memory

If only one memory unit exists, one instruction must wait.

Your simulation should compare:

Without hazard handling
↓
More stalls

↓
More clock cycles
↓
Lower performance

With forwarding / prediction
↓
Fewer stalls
↓
Fewer clock cycles
↓
Higher performance

Code:

```
#include <iostream>
using namespace std;

int main() {

    cout << "===== PIPELINE HAZARD SIMULATION =====\n";

    cout << "\nInstructions:\n";
    cout << "I1: ADD R1, R2, R3\n";
    cout << "I2: SUB R4, R1, R5\n";
    cout << "I3: BEQ R4, R0, LABEL\n";
    cout << "I4: LOAD R6, 0(R7)\n";

    cout << "\n1. DATA HAZARD\n";
    cout << "I2 needs R1 produced by I1." << endl;
    cout << "Without forwarding: Pipeline Stall required." << endl;

    cout << "\nApplying Data Forwarding..." << endl;
    cout << "Result of I1 forwarded directly to I2." << endl;
    cout << "Stall cycles reduced." << endl;

    cout << "\n2. CONTROL HAZARD\n";
    cout << "I3 is a branch instruction." << endl;
    cout << "Branch decision is not immediately known." << endl;

    cout << "\nApplying Branch Prediction..." << endl;
    cout << "Prediction: Branch NOT Taken." << endl;
    cout << "Pipeline continues fetching instructions." << endl;

    cout << "\n3. STRUCTURAL HAZARD\n";
```

```
    cout << "I4 requires memory access." << endl;
    cout << "Instruction fetch also requires memory." << endl;
    cout << "Both compete for the same resource." << endl;

    cout << "\nApplying Stall..." << endl;
    cout << "One instruction is delayed by one cycle." << endl;

    cout << "\n===== HAZARD RESOLUTION =====\n";

    cout << "Data Hazard      -> Data Forwarding" << endl;
    cout << "Control Hazard   -> Branch Prediction" << endl;
    cout << "Structural Hazard-> Pipeline Stall" << endl;

    cout << "\nHazard resolution improves pipeline performance."
         << endl;

    return 0;
}
```

Output:

```
===== PIPELINE HAZARD SIMULATION =====

Instructions:
I1: ADD R1, R2, R3
I2: SUB R4, R1, R5
I3: BEQ R4, R0, LABEL
I4: LOAD R6, 0(R7)

1. DATA HAZARD
I2 needs R1 produced by I1.
Without forwarding: Pipeline Stall required.

Applying Data Forwarding...
Result of I1 forwarded directly to I2.
Stall cycles reduced.

2. CONTROL HAZARD
I3 is a branch instruction.
Branch decision is not immediately known.

Applying Branch Prediction...
Prediction: Branch NOT Taken.
Pipeline continues fetching instructions.

3. STRUCTURAL HAZARD
I4 requires memory access.
Instruction fetch also requires memory.
Both compete for the same resource.

Applying Stall...
One instruction is delayed by one cycle.

===== HAZARD RESOLUTION =====
Data Hazard      -> Data Forwarding
Control Hazard   -> Branch Prediction
Structural Hazard-> Pipeline Stall

Hazard resolution improves pipeline performance.


=== Code Execution Successful ===
```

5. Superscalar, Out-of-Order, Speculative Execution and SMT

This is the most advanced of the five topics.

A superscalar processor can execute multiple instructions in the same clock cycle.

For example, a 2-wide processor can potentially execute:

Cycle 1 → I1 + I2
Cycle 2 → I3 + I4
Cycle 3 → I5 + I6

Instead of:

Cycle 1 → I1
Cycle 2 → I2
Cycle 3 → I3
Cycle 4 → I4

Out-of-order execution

Instructions don't necessarily execute in the order they appear.

Example:

I1 → waiting for memory
I2 → ready
I3 → ready

Instead of waiting for I1:

I1 → I2 → I3

the processor can execute:

I2 → I3 → I1

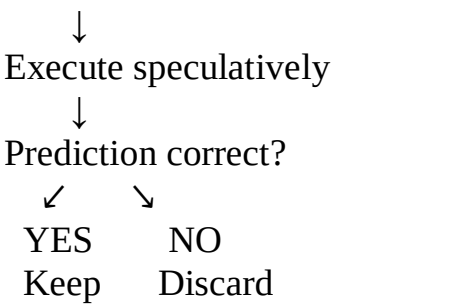
when dependencies allow it.

Speculative execution

The processor predicts what instructions will be needed next and executes them before knowing with certainty that they are required.

Example:

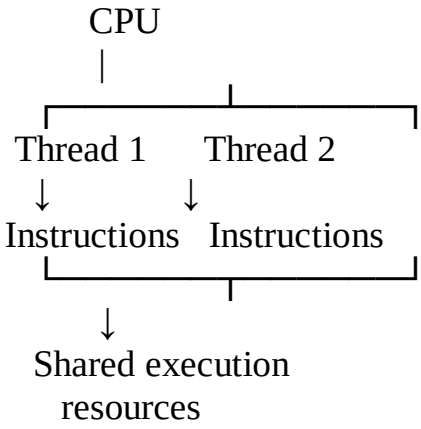
Branch prediction
↓
Predict branch
↓



Hyper-threading / SMT

Simultaneous Multithreading (SMT) allows multiple instruction streams/threads to share processor resources.

For example:



You can compare:

Architecture	Parallelism	Complexity	Expected throughput
Single issue	Low	Low	Low
Pipelined	Medium	Medium	Higher
Superscalar	High	High	Higher
Out-of-order	Very high	Very high	Very high
SMT	Multiple threads	Very high	Higher resource utilization

Code:

```
#include <iostream>
using namespace std;
```

```
int main() {

    cout << "===== SUPERSCALAR PROCESSOR =====\n\n";

    int instructions = 8;
    int issueWidth = 2;

    cout << "Total Instructions = "
         << instructions << endl;

    cout << "Superscalar Issue Width = "
         << issueWidth << endl;

    cout << "\nIn-Order Execution:\n";

    int inOrderCycles = instructions;

    for (int i = 1; i <= instructions; i++) {
        cout << "Cycle " << i
             << ": Execute Instruction I"
             << i << endl;
    }

    cout << "\nOut-of-Order Execution:\n";

    cout << "Cycle 1: I1, I2" << endl;
    cout << "Cycle 2: I4, I5" << endl;
    cout << "Cycle 3: I3, I6" << endl;
    cout << "Cycle 4: I7, I8" << endl;

    int outOfOrderCycles = 4;

    cout << "\nSpeculative Execution:\n";
    cout << "Branch prediction is performed." << endl;
    cout << "Instructions after predicted branch are executed." << endl;

    cout << "\nHyper-Threading / SMT:\n";
```

```
    cout << "Thread 1 executes instructions." << endl;
    cout << "Thread 2 executes instructions simultaneously." << endl;

    int smtCycles = 2;

    cout << "\n===== PERFORMANCE =====\n";

    cout << "In-Order Cycles = "
         << inOrderCycles << endl;

    cout << "Superscalar Out-of-Order Cycles = "
         << outOfOrderCycles << endl;

    cout << "SMT Estimated Cycles = "
         << smtCycles << endl;

    double speedup =
        (double)inOrderCycles / outOfOrderCycles;

    cout << "Superscalar Speedup = "
         << speedup << "x" << endl;

    cout << "\nResource utilization is improved because"
         << " multiple instructions can execute in parallel."
         << endl;

    return 0;
}
```

Output:

```
===== SUPERSCALAR PROCESSOR =====

Total Instructions = 8
Superscalar Issue Width = 2

In-Order Execution:
Cycle 1: Execute Instruction I1
Cycle 2: Execute Instruction I2
Cycle 3: Execute Instruction I3
Cycle 4: Execute Instruction I4
Cycle 5: Execute Instruction I5
Cycle 6: Execute Instruction I6
Cycle 7: Execute Instruction I7
Cycle 8: Execute Instruction I8

Out-of-Order Execution:
Cycle 1: I1, I2
Cycle 2: I4, I5
Cycle 3: I3, I6
Cycle 4: I7, I8

Speculative Execution:
Branch prediction is performed.
Instructions after predicted branch are executed.

Hyper-Threading / SMT:
Thread 1 executes instructions.
Thread 2 executes instructions simultaneously.

===== PERFORMANCE =====
In-Order Cycles = 8
Superscalar Out-of-Order Cycles = 4
SMT Estimated Cycles = 2
Superscalar Speedup = 2x

Resource utilization is improved because multiple instructions can execute in parallel.


=== Code Execution Successful ===
```